ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

Студент: Александров Ярослав Дмитриевич
Группа: 25212

Тема: Изучение оптимизирующего компилятора GCC и исследование влияния уровней оптимизации на время исполнения программы.

1. ЦЕЛЬ РАБОТЫ

1. Знакомство с программной архитектурой x86/x86-64.
2. Анализ ассемблерного листинга программы для архитектуры x86/x86-64.

2. ВАРИАНТ ЗАДАНИЯ

Реализуемый алгоритм: Алгоритм вычисления числа Пи с помощью разложения в ряд (ряд Грегори-Лейбница) по формуле Лейбница N первых членов ряда:

$$\pi = 4\sum_{n=0}^{\infty}\frac{(-1)^n}{2n+1} = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \ldots + 4\frac{(-1)^n}{(2n+1)} + \ldots$$

Параметры исследования:

- Архитектура: x86 (x86_64).
- Команда компиляции (базовая): `g++ lab3.cpp -o lab3.exe`
- Команда запуска: `.\lab3.exe`

4. ЛИСТИНГ ПРОГРАММЫ

```
#include <iostream>
#include <math.h>
#include <iomanip>
#include <windows.h>

long double sumc(long long n){
```

```
    long double pi = 0;

    for (long long i = 0; i <= n; i++){
        if(i % 2 == 0)
            pi += 4*((long double)(1.0/(2.0*i+1)));
        else
            pi -= 4*((long double)(1.0/(2.0*i+1)));
    }
    return pi;
}

int main(){
    long long n;
    std::cin >> n;

    long double pi = sumc(n);



    std::cout << std::setprecision(15) << pi;

    return 0;
}
```

5. Описание работы.

Свой код из первой лабораторной работы переделал в ассемблерный листинг с различными уровнями оптимизации.

x86-64 –O0

```
# === Функция setprecision (вспомогательная) ===
.section    .text$_ZSt12setprecisioni,"x"
.linkonce discard
.globl  _ZSt12setprecisioni
.def    _ZSt12setprecisioni;    .scl    2;  .type   32; .endef
.seh_proc   _ZSt12setprecisioni
_ZSt12setprecisioni:
.LFB3499:
pushq   %rbp                        # Сохраняем базовый указатель вызывающей
функции
.seh_pushreg    %rbp
```

```
movq    %rsp, %rbp                  # Устанавливаем новый базовый указатель
кадра
.seh_setframe   %rbp, 0
.seh_endprologue
movl    %ecx, 16(%rbp)              # Сохраняем аргумент (точность) в стек
movl    16(%rbp), %eax              # Загружаем аргумент в регистр возврата
popq    %rbp                        # Восстанавливаем базовый указатель
ret                                 # Возврат (результат в %eax)
.seh_endproc

.text

# === Функция sumc: вычисление числа π по ряду Грегори-Лейбница ===
.globl  _Z4sumcx
.def    _Z4sumcx;   .scl    2;  .type   32; .endef
.seh_proc   _Z4sumcx
_Z4sumcx:
.LFB9955:
pushq   %rbp                        # Пролог функции: сохраняем %rbp
.seh_pushreg    %rbp
movq    %rsp, %rbp                  # Устанавливаем базовый указатель текущего
кадра
.seh_setframe   %rbp, 0
subq    $32,  %rsp                  # Выделяем 32 байта локальной памяти в
стеке
.seh_stackalloc 32
.seh_endprologue

# Сохранение аргументов функции
movq    %rcx, 16(%rbp)              # 1-й аргумент: указатель на результат
(куда записать pi)
movq    %rdx, 24(%rbp)              # 2-й аргумент: n (количество членов ряда)

# Инициализация: long double pi = 0;
movq    16(%rbp), %rax              # Загружаем адрес для записи результата
fldz                                # Загружаем 0.0 в стек FPU (st(0))
fstpt   (%rax)                      # Сохраняем pi = 0 по адресу результата
(80-бит long double)

# Инициализация цикла: for (long long i = 0; ...)
movq    $0, -8(%rbp)                # i = 0 (счетчик цикла в стеке)
jmp .L4                           # Переход к проверке условия цикла

# === ТЕЛО ЦИКЛА ===
.L7:
# Проверка условия: if (i % 2 == 0)
```

```
movq    -8(%rbp), %rax              # Загружаем текущее значение i
andl    $1, %eax                    # Вычисляем i % 2 (побитовое И с 1)
testq   %rax, %rax                  # Проверяем результат на ноль
jne .L5                            # Если i нечетное (i % 2 != 0), переходим к
ветке else

# === Ветка IF (i четное): pi += 4 * (1.0 / (2.0*i + 1.0)) ===
# Вычисление знаменателя (2.0 * i + 1.0) через SSE (double)
pxor    %xmm0, %xmm0                # Обнуляем регистр %xmm0
cvtsi2sdq   -8(%rbp), %xmm0         # Конвертируем i (int64) в double
movapd  %xmm0, %xmm1                # Копируем i в %xmm1
addsd   %xmm0, %xmm1                # %xmm1 = i + i = 2.0 * i
movsd   .LC1(%rip), %xmm0           # Загружаем константу 1.0
addsd   %xmm0, %xmm1                # %xmm1 = 2.0*i + 1.0

# Вычисление дроби: 1.0 / (2.0*i + 1.0)
movsd   .LC1(%rip), %xmm0           # Снова загружаем 1.0
divsd   %xmm1, %xmm0                # %xmm0 = 1.0 / (2.0*i + 1.0)
movsd   %xmm0, -24(%rbp)            # Сохраняем промежуточный результат в стек

# Переход к x87 для работы с long double (80 бит)
fldl    -24(%rbp)                   # Загружаем double из стека в st(0)
fldt    .LC2(%rip)                  # Загружаем константу 4.0 (80-бит long
double)
fmulp   %st, %st(1)                 # st(1) = st(1) * st(0) = 4.0 *
(1.0/(2*i+1))

# Прибавление к pi: pi += вычисленное значение
movq    16(%rbp), %rax              # Загружаем адрес переменной pi
fldt    (%rax)                      # Загружаем текущее значение pi в st(0)
faddp   %st, %st(1)                 # st(1) = st(1) + st(0) = pi + вычисленное
movq    16(%rbp), %rax              # Снова загружаем адрес pi
fstpt   (%rax)                      # Сохраняем новое значение pi в память
jmp .L6                            # Переход к шагу цикла (пропускаем ветку
else)

# === Ветка ELSE (i нечетное): pi -= 4 * (1.0 / (2.0*i + 1.0)) ===
.L5:
movq    16(%rbp), %rax              # Загружаем адрес pi
fldt    (%rax)                      # Загружаем текущее значение pi

# Вычисление 4.0 / (2.0*i + 1.0) (идентично ветке if)
pxor    %xmm0, %xmm0
cvtsi2sdq   -8(%rbp), %xmm0
movapd  %xmm0, %xmm1
addsd   %xmm0, %xmm1
```

```
movsd   .LC1(%rip), %xmm0
addsd   %xmm0, %xmm1
movsd   .LC1(%rip), %xmm0
divsd   %xmm1, %xmm0
movsd   %xmm0, -24(%rbp)
fldl    -24(%rbp)
fldt    .LC2(%rip)
fmulp   %st, %st(1)

# Вычитание из pi: pi -= вычисленное значение
fsubrp  %st, %st(1)                # st(1) = st(1) - st(0) = pi - вычисленное
movq    16(%rbp), %rax             # Загружаем адрес pi
fstpt   (%rax)                     # Сохраняем обновленное значение pi

# === ШАГ ЦИКЛА: i++ ===
.L6:
addq    $1, -8(%rbp)               # Инкремент счетчика цикла: i = i + 1

# === ПРОВЕРКА УСЛОВИЯ ЦИКЛА: i <= n ===
.L4:
movq    -8(%rbp), %rax             # Загружаем текущее значение i
cmpq    24(%rbp), %rax             # Сравниваем i и n (вычисляет i - n)
jle .L7                           # Если i <= n, продолжаем цикл (переход к
.L7)

# === ВОЗВРАТ РЕЗУЛЬТАТА: return pi; ===
movq    16(%rbp), %rax             # Загружаем адрес результата
fldt    (%rax)                     # Загружаем финальное значение pi в st(0)
movq    16(%rbp), %rax
fstpt   (%rax)                     # Сохраняем результат (избыточная операция
при -O0)
movq    16(%rbp), %rax
addq    $32, %rsp                  # Освобождаем локальный стек
popq    %rbp                       # Восстанавливаем базовый указатель
ret                                # Возврат (результат в st(0))
.seh_endproc

# === Функция main ===
.globl  main
.def    main;   .scl    2;  .type   32; .endef
.seh_proc   main
main:
.LFB9956:
pushq   %rbp                       # Пролог main
.seh_pushreg    %rbp
movq    %rsp, %rbp
.seh_setframe   %rbp, 0
```

```
subq    $160, %rsp                  # Выделяем 160 байт для локальных
переменных
.seh_stackalloc 160
.seh_endprologue
call    __main                      # Инициализация runtime среды

# std::cin >> n;
leaq    -72(%rbp), %rdx             # Адрес переменной n
movq    .refptr._ZSt3cin(%rip), %rax # Адрес объекта std::cin
movq    %rax, %rcx                  # 1-й аргумент: std::cin
call    _ZNSirsERx                  # Вызов operator>>(long long&)

# Вызов функции sumc(n)
movq    -72(%rbp), %rdx             # Загружаем значение n
leaq    -112(%rbp), %rax            # Адрес для результата
movq    %rax, %rcx                  # 1-й аргумент: указатель на результат
call    _Z4sumcx                    # Вызов функции sumc
fldt    -112(%rbp)                  # Загружаем результат (long double)
fstpt   -16(%rbp)                   # Сохраняем в локальную переменную pi

# std::cout << std::setprecision(15) << pi;
movl    $15, %ecx                   # Аргумент: точность 15
call    _ZSt12setprecisioni         # Вызов setprecision(15)
movl    %eax, %edx                  # Результат (манипулятор) в %edx
movq    .refptr._ZSt4cout(%rip), %rax # Адрес std::cout
movq    %rax, %rcx                  # 1-й аргумент: std::cout
call
_ZStlsIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_St13_Setprecision

# Вывод значения pi
movq    %rax, %rcx                  # Поток cout (возвращенный предыдущим
вызовом)
fldt    -16(%rbp)                   # Загружаем pi
fstpt   -112(%rbp)                  # Сохраняем во временную ячейку
leaq    -112(%rbp), %rax
movq    %rax, %rdx                  # 2-й аргумент: указатель на pi
call    _ZNSolsEe                   # Вызов operator<<(long double)

# Возврат из main
movl    $0, %eax                    # return 0
addq    $160, %rsp                  # Освобождаем стек
popq    %rbp                        # Восстанавливаем %rbp
ret                                 # Возврат
.seh_endproc

# === Секция данных: константы ===
```

```
.section .rdata, "dr"
.align 8
.LC1:                                   # Константа 1.0 (double, 64 бита)
.long   0
.long   1072693248                      # IEEE 754: 0x3FF0000000000000

.align 16
.LC2:                                   # Константа 4.0 (long double, 80 бит)
.long   0
.long   -2147483648                     # Мантисса
.long   16385                           # Экспонента: 2^2 * 1.0 = 4.0
.long   0
```

x86-64 –O3

```
# === Функция sumc с максимальной оптимизацией ===
.p2align 4
.globl  _Z4sumcx
.def    _Z4sumcx;   .scl    2;  .type   32; .endef
.seh_proc   _Z4sumcx
_Z4sumcx:
.LFB11720:
subq    $24, %rsp                       # Выделяем только 24 байта (оптимизация: нет %rbp)
.seh_stackalloc 24
.seh_endprologue

movq    %rcx, %rax                      # Сохраняем указатель на результат в %rax
testq   %rdx, %rdx                      # Проверяем: если n < 0
js  .L9                                 # Переход к обработке ошибки (возврат 0)

# Инициализация: pi = 0, подготовка констант
fldz                                    # Загружаем 0.0 в st(0) (начальное значение pi)
addq    $1, %rdx                        # Оптимизация: n++ (условие i < n+1 вместо i <= n)
flds    .LC2(%rip)                      # Загружаем константу 4.0 ОДИН РАЗ в st(1)
xorl    %r8d, %r8d                      # i = 0 в регистре %r8d (быстрее, чем mov)
movsd   .LC1(%rip),  %xmm1              # Загружаем константу 1.0 ОДИН РАЗ в %xmm1
jmp .L6                                 # Переход к телу цикла

# Выравнивание для оптимизации кэша инструкций
```

```
.p2align 6
.p2align 4,,10
.p2align 3

# === Ветка НЕЧЕТНОГО i (pi -= ...) ===
.L11:
addq    $1, %r8                     # i++ (инкремент в регистре)
faddp   %st, %st(2)                 # st(2) = st(2) + st(0) (добавляем к pi)
cmpq    %rdx, %r8                   # Сравниваем i и n+1
je  .L12                            # Если i == n+1, выходим из цикла

# === ТЕЛО ЦИКЛА (общая часть для четного и нечетного) ===
.L6:
pxor    %xmm0, %xmm0                # Обнуляем %xmm0
fld %st(0)                          # Дублируем st(0) (4.0) для вычислений
movapd  %xmm1, %xmm2                # Копируем 1.0 в %xmm2
cvtsi2sdq   %r8,  %xmm0             # Конвертируем i (int64) в double
addsd   %xmm0, %xmm0                # %xmm0 = 2.0 * i
addsd   %xmm1, %xmm0                # %xmm0 = 2.0*i + 1.0 (используем
сохраненную 1.0)
divsd   %xmm0, %xmm2                # %xmm2 = 1.0 / (2.0*i + 1.0)
movsd   %xmm2, 8(%rsp)              # Временно сохраняем в стек
fmull   8(%rsp)                     # Умножаем st(0) на результат (4.0 * ...)

testb   $1, %r8b                    # Проверяем четность i (младший бит)
je  .L11                            # Если четное, переходим к ветке сложения

# Ветка НЕЧЕТНОГО (вычитание)
addq    $1, %r8                     # i++
fsubrp  %st, %st(2)                 # st(2) = st(2) - st(0) (вычитаем из pi)
cmpq    %rdx, %r8                   # Сравниваем i и n+1
jne .L6                             # Если не равно, продолжаем цикл

fst p   %st(0)                      # Очищаем стек FPU
jmp .L10                            # Переход к возврату

# Выравнивание
.p2align 4,,10
.p2align 3

# Выход из цикла при i == n+1
.L12:
fstp    %st(0)                      # Очищаем стек FPU

# Возврат результата
.L10:
```

```
fstpt   (%rax)                      # Сохраняем pi по указателю результата
addq    $24, %rsp                   # Освобождаем стек
ret                                 # Возврат

# Обработка ошибки: n < 0
.p2align 4,,10
.p2align 3
.L9:
movq    $0, (%rcx)                  # Записываем 0 в младшие 64 бита
результата
movl    $0, 8(%rcx)                 # Записываем 0 в старшие 16 бит
addq    $24, %rsp
r et                                # Возврат
.seh_endproc

# === Функция main с инлайнгом sumc ===
.section    .text.startup, "x"
.p2align 4
.globl  main
.def    main;   .scl    2;  .type   32; .endef
.seh_proc   main
main:
.LFB11721:
subq    $88, %rsp                   # Выделяем 88 байт стека
.seh_stackalloc 88
.seh_endprologue
call    __main                      # Инициализация runtime

# std::cin >> n;
movq    .refptr._ZSt3cin(%r ip), %rcx # Адрес std::cin
leaq    72(%rsp), %rdx              # Адрес переменной n
call     ZNSi10_M_extractIxEERSiRT  # Вызов operator>>

# Проверка n и подготовка к циклу (инлайнг sumc)
movq    72(%rsp), %rdx              # Загружаем n
testq   %rdx, %rdx                  # Проверяем знак
js  .L18                           # Если n < 0, переход к обработке

fldz                                # pi = 0
movsd   .LC1(%rip), %xmm1           # 1.0 в %xmm1
addq    $1, %rdx                    # n++ (оптимизация условия)
xorl    %eax, %eax                  # i = 0 в %eax
flds    .LC2(%rip)                  # 4.0 в st(0)
jmp .L17                           # Переход к телу цикла

# Выравнивание
```

```
.p2align 6
.p2align 4,,10
.p2align 3

# Ветка ЧЕТНОГО i (сложение)
.L20:
addq    $1 , %rax                   # i++
faddp   %st, %st(2)                 # pi += вычисленное
cmpq    %rdx, %rax                  # Сравнение i и n+1
je  .L21                           # Выход если равно

# Тело цикла (общая часть)
.L17:
pxor    %xmm0, %xmm0
fld %st(0)
movapd  %xmm1, %xmm2
cvtsi2sdq   %rax, %xmm0             # i → double
addsd   %xmm0, %xmm0                # 2*i
addsd   %xmm1, %xmm0                # 2*i + 1
divsd   %xmm0, %xm m2               # 1.0 / (2*i+1)
movsd   %xmm2, 40(%rsp)             # Временное сохранение
fmull   40(%rsp)                    # Умножение на 4.0

testb   $1, %al                     # Проверка четности
je  .L20                           # Если четное, переход к сложению

# Ветка НЕЧЕТНОГО (вычитание)
addq    $1, %rax                    # i++
fsubrp  %st, %st(2)                 # pi -= вычисленное
cmpq    %rdx, %rax                  # Сравнение
jne .L17                           # Продолжение цикла

fstp    %st(0)                      # Очистка стека
jmp .L14                           # Переход к выводу

# Выход из цикла
.L21:
fstp    %st(0)

# Вывод результата
.L14:
movq    .refptr._Z St4cout(%rip), %rcx # Адрес std::cout
leaq    48(%rsp), %rdx              # Адрес для результата
movq    (%rcx), %rax
movq    -24(%rax), %rax
movq    $15, 8(%rax,%rcx)           # setprecision(15) — инлайнинг!
```

```
fstpt   48(%rsp)                    # Сохраняем pi
call     ZNSo9_M_insertIeEERSoT     # Вывод long double

xorl    %eax, %eax                  # return 0
addq    $88, %rsp                   # Освобождаем стек
ret                                 # Возврат

# Обработка n < 0
.L18:
fldz                                # pi = 0
jmp .L14                           # Переход к выводу
.seh_endproc

# === Константы ===
.section .rdata, "dr"
.align 8
.LC1:                               # 1.0 (double)
.long   0
.long   1072693248

.align 4
.LC2:                               # 4.0 (float для flds)
.long   1082130432                  # IEEE 754: 0x40800000
```

x86-64 –Og

```

# ФУНКЦИЯ sumc
.globl  _Z4sumcx
.def    _Z4sumcx;   .scl    2;  .type   32; .endef
.seh_proc   _Z4sumcx
_Z4sumcx:
.LFB11720:
# --- ПРОЛОГ ФУНКЦИИ ---
subq    $24, %rsp                   # Выделяем 24 байта локальной памяти в
стеке
```

```
                                    # (оптимизация: не используем %rbp как
базовый указатель)
.seh_stackalloc 24
.seh_endprologue

movq    %rcx, %rax                  # Сохраняем указатель на результат (1-й
аргумент) в %rax
                                    # %rcx содержит адрес, куда нужно записать
возвращаемое значение pi

# --- ИНИЦИАЛИЗАЦИЯ: long double pi = 0; ---
fldz                                # Загружаем константу 0.0 в стек FPU
(st(0))
                                    # Это начальное значение переменной pi
fstpt   (%rcx)                      # Сохраняем pi = 0 по адресу результата
(80-бит long double)
                                    # fstpt = Float Store Ten-byte and Pop

# --- ИНИЦИАЛИЗАЦИЯ ЦИКЛА: for (long long i = 0; ...) ---
movl    $0, %r8d                    # i = 0 в регистре %r8d (младшие 32 бита
регистра %r8)
                                    # Оптимизация: переменная цикла хранится в
регистре, а не в стеке
jmp .L2                            # Безусловный переход к проверке условия
цикла

# ВЕТКА ELSE (i нечётное): pi -= 4 * (1.0 / (2.0*i + 1.0))
.L3:
# --- Вычисление знаменателя (2.0 * i + 1.0) через SSE (double) ---
pxor    %xmm0, %xmm0                # Обнуляем регистр %xmm0 (быстрее, чем
mov)
cvtsi2sdq   %r8, %xmm0              # Конвертируем i (int64 из %r8) в double и
сохраняем в %xmm0
                                    # cvtsi2sdq = Convert Signed Integer to
Scalar Double Quadword
addsd   %xmm0, %xmm0                # %xmm0 = %xmm0 + %xmm0 = 2.0 * i
                                    # addsd = Add Scalar Double (складываем
только младшие 64 бита)
movsd   .LC1(%rip), %xmm1           # Загружаем константу 1.0 (double) из
секции данных
                                    # movsd = Move Scalar Double, %rip = RIP-
относительная адресация
addsd   %xmm1, %xmm0                # %xmm0 = 2.0*i + 1.0

# --- Вычисление дроби: 1.0 / (2.0*i + 1.0) ---
divsd   %xmm0, %xmm1                # %xmm1 = %xmm1 / %xmm0 = 1.0 / (2.0*i +
1.0)
```

```
                                    # divsd = Divide Scalar Double
movsd   %xmm1, 8(%rsp)              # Сохраняем промежуточный результат
(double) в стек
                                    # Нужно для перехода от SSE к x87 FPU

# --- Переход к x87 для работы с long double (80 бит) ---
flds    .LC2(%rip)                  # Загружаем константу 4.0 (float, 32 бита)
в st(0)
                                    # flds = Float Load Single
fmull   8(%rsp)                     # Умножаем st(0) на значение из памяти:
st(0) = 4.0 * (1.0/(2*i+1))
                                    # fmull = Float Multiply Long (32-бит
float)

# --- Вычитание из pi: pi -= вычисленное значение ---
fldt    (%rax)                      # Загружаем текущее значение pi из памяти
в st(0)
                                    # fldt = Float Load Ten-byte (80-бит long
double)
                                    # Теперь st(0) = pi, st(1) = 4.0/(2*i+1)
fsubp   %st, %st(1)                 # st(1) = st(1) - st(0) = pi - вычисленное
                                    # fsubp = Float Subtract and Pop (удаляем
st(0))
                                    # Результат в st(0) = новое значение pi
fstpt   (%rax)                      # Сохраняем обновлённое pi обратно в
память
                                    # fstpt = Float Store Ten-byte and Pop

# ШАГ ЦИКЛА: i++
.L4:
addq    $1, %r8                     # Инкремент счётчика цикла: i = i + 1
                                    # Работаем напрямую с регистром %r8
(быстро!)

# ПРОВЕРКА УСЛОВИЯ ЦИКЛА: i <= n
.L2:
cmpq    %rdx, %r8                   # Сравниваем i (%r8) и n (%rdx)
                                    # cmpq = Compare Quadword (вычисляет %r8 -
%rdx, обновляет флаги)
jg  .L6                            # Jump if Greater: если i > n, выходим из
цикла (переход к .L6)

# --- Проверка чётности: if (i % 2 == 0) ---
testb   $1, %r8b                    # Проверяем младший бит %r8 (эквивалент i
% 2)
                                    # testb = Test Byte (побитовое И с 1,
обновляет флаги)
```

```
                                    # %r8b = младший байт регистра %r8
jne .L3                             # Jump if Not Equal: если бит = 1
(нечётное), переход к ветке else (.L3)

# ВЕТКА IF (i чётное): pi += 4 * (1.0 / (2.0*i + 1.0))
# Вычисление знаменателя (2.0 * i + 1.0) через SSE (double)
pxor    %xmm0, %xmm0                # Обнуляем %xmm0
cvtsi2sdq   %r8, %xmm0              # Конвертируем i (int64) в double
addsd   %xmm0, %xmm0                # %xmm0 = 2.0 * i
movsd   .LC1(%rip), %xmm1           # Загружаем константу 1.0
addsd   %xmm1, %xmm0                # %xmm0 = 2.0*i + 1.0

# Вычисление дроби: 1.0 / (2.0*i + 1.0)
divsd   %xmm0, %xmm1                # %xmm1 = 1.0 / (2.0*i + 1.0)
movsd   %xmm1, 8(%rsp)              # Сохраняем в стек

# Переход к x87 для работы с long double
flds    .LC2(%rip)                  # Загружаем 4.0
fmull   8(%rsp)                     # Умножаем: 4.0 * (1.0/(2*i+1))

# Сложение с pi: pi += вычисленное значение
fldt    (%rax)                      # Загружаем текущее pi
faddp   %st, %st(1)                 # st(1) = st(1) + st(0) = pi + вычисленное
                                    # faddp = Float Add and Pop
fstpt   (%rax)                      # Сохраняем новое pi в память
jmp .L4                            # Переход к шагу цикла (i++)

# ВОЗВРАТ ИЗ ФУНКЦИИ
.L6:
addq    $24, %rsp                   # Освобождаем локальный стек (24 байта)
ret                                 # Возврат в вызывающую функцию
                                    # Результат (pi) уже сохранён в память по
адресу %rax
.seh_endproc

# ФУНКЦИЯ main
.globl  main
.def    main;   .scl    2;  .type   32; .endef
.seh_proc   main
main:
.LFB11721:
# --- ПРОЛОГ ФУНКЦИИ ---
pushq   %rbx                        # Сохраняем callee-saved регистр %rbx
                                    # (будем использовать его для хранения
адреса результата)
.seh_pushreg    %rbx
```

```
subq    $64, %rsp                   # Выделяем 64 байта локальной памяти
.seh_stackalloc 64
.seh_endprologue
call    __main                      # Инициализация runtime среды MinGW

# --- std::cin >> n; ---
leaq    56(%rsp), %rdx              # Вычисляем адрес переменной n (смещение
56 от %rsp)
                                    # leaq = Load Effective Address (q =
quadword, 64 бита)
movq    .refptr._ZSt3cin(%rip), %rcx # Загружаем адрес объекта std::cin
                                    # .refptr = ссылка на глобальный объект
                                    # %rip = RIP-относительная адресация
call    _ZNSi10_M_extractIxEERSiRT  # Вызов operator>>(long long&) для чтения
n
                                    # Результат: n сохранено по адресу
56(%rsp)

# --- Вызов функции sumc(n) ---
leaq    32(%rsp), %rbx              # Вычисляем адрес для результата
(переменная pi)
                                    # Сохраняем в %rbx (callee-saved регистр)
movq    56(%rsp), %rdx              # Загружаем значение n из стека в %rdx (2-
й аргумент)
movq    %rbx, %rcx                  # Помещаем адрес результата в %rcx (1-й
аргумент)
call    _Z4sumcx                    # Вызов функции sumc(n)
                                    # Результат записан по адресу, хранящемуся
в %rbx

# --- std::cout << std::setprecision(15) << pi; ---
fldt    32(%rsp)                    # Загружаем результат pi (long double) из
памяти в st(0)
movq    .refptr._ZSt4cout(%rip), %rcx # Загружаем адрес объекта std::cout
movq    (%rcx), %rax                # Разыменовываем указатель на cout
movq    %rcx, %rdx                  # Сохраняем адрес cout в %rdx
addq    -24(%rax), %rdx             # Вычисляем адрес внутреннего буфера
форматирования cout
movq    $15, 8(%rdx)                # Устанавливаем точность 15 (инлайнинг
setprecision!)
                                    # Оптимизация: вместо вызова функции
setprecision(15)
                                    # компилятор напрямую записывает значение
15 в поле объекта cout
fstpt   32(%rsp)                    # Сохраняем pi обратно в память
(подготовка к выводу)
```

```
movq    %rbx, %rdx                  # Помещаем адрес pi в %rdx (2-й аргумент
для operator<<)
call    _ZNSo9_M_insertIeEERSoT     # Вызов operator<<(long double) для вывода
pi

# --- Возврат из main ---
movl    $0, %eax                    # return 0 (код успешного завершения)
addq    $64, %rsp                   # Освобождаем локальный стек
popq    %rbx                        # Восстанавливаем callee-saved регистр
%rbx
ret                                 # Возврат из main
.seh_endproc

# СЕКЦИЯ ДАННЫХ: константы
.section .rdata, "dr"
.align 8
.LC1:                               # Константа 1.0 (тип double, 64 бита)
.long   0                           # Младшие 32 бита: 0x00000000
.long   1072693248                  # Старшие 32 бита: 0x3FF00000 (IEEE 754)

.align 4
.LC2:                               # Константа 4.0 (тип float, 32 бита)
.long   1082130432                  # IEEE 754: 0x40800000

# Ссылки на глобальные объекты стандартной библиотеки
.section    .rdata$.refptr._ZSt4cout, "dr"
.p2align    3, 0
.globl  .refptr._ZSt4cout
.linkonce   discard
.refptr._ZSt4cout:
.quad   _ZSt4cout                   # Адрес объекта std::cout

.section    .rdata$.refptr._ZSt3cin, "dr"
.p2align    3, 0
.globl  .refptr._ZSt3cin
.linkonce   discard
.refptr._ZSt3cin:
.quad   _ZSt3cin                    # Адрес объекта std::cin
```

x86-64 -m32 -O3

```
# ФУНКЦИЯ sumc
.p2align 4                          # Выравнивание по 16 байт для оптимизации
кэша
.globl  __Z4sumcx
.def    __Z4sumcx;  .scl    2;  .type   32; .endef
__Z4sumcx:
LFB4663:
.cfi_startproc
# --- ПРОЛОГ ФУНКЦИИ ---
pushl   %ebp                        # Сохраняем старый базовый указатель
.cfi_def_cfa_offset 8
.cfi_offset 5, -8
movl    %esp, %ebp                  # Устанавливаем новый базовый указатель
.cfi_def_cfa_register 5
pushl   %edi                        # Сохраняем callee-saved регистры
pushl   %esi                        # (%edi, %esi, %ebx будут использоваться
для переменных)
pushl   %ebx
andl    $-8, %esp                   # Выравнивание стека по 8 байт (критично
для x87/SSE)
subl    $16, %esp                   # Выделяем 16 байт локальной памяти
.cfi_offset 7, -12
.cfi_offset 6, -16
.cfi_offset 3, -20

# --- Загрузка аргумента n (64-битное число в двух 32-битных частях) ---
movl    12(%ebp), %edi              # Старшие 32 бита n → %edi
movl    8(%ebp), %esi               # Младшие 32 бита n → %esi
                                    # В 32-бит архитектуре long long
разбивается на две части

# --- Проверка: если n < 0, возвращаем 0 ---
testl   %edi, %edi                  # Проверяем старшую часть n на знак
js  L6                             # Jump if Sign: если n отрицательное,
переход к возврату 0

# --- Инициализация: n++, pi = 0, i = 0 ---
addl    $1, %esi                    # n_low++ (оптимизация: условие i < n+1
вместо i <= n)
fldz                                # Загружаем 0.0 в st(0) (начальное
значение pi)
adcl    $0, %edi                    # n_high += carry (если был перенос от
сложения младшей части)
                                    # adcl = Add with Carry Long
xorl    %eax, %eax                  # i_low = 0 (обнуление через XOR быстрее,
чем mov)
xorl    %edx, %edx                  # i_high = 0
```

```
jmp L5                          # Переход к телу цикла

# Выравнивание для оптимизации кэша инструкций
.p2align 4,,10
.p2align 3

# ВЕТКА ЧЁТНОГО i (сложение): pi += 4 * (1.0 / (2.0*i + 1.0))
L10:
faddp   %st, %st(1)             # st(1) = st(1) + st(0) = pi + вычисленное
                                # faddp = Float Add and Pop
                                # Результат в st(0) = новое значение pi

# ШАГ ЦИКЛА: i++ (64-битный инкремент через пару регистров)
L4:
addl    $1, %eax                # i_low++ (инкремент младшей части)
adcl    $0, %edx                # i_high += carry (инкремент старшей части
с учётом переноса)
                                # Это эмуляция 64-битного инкремента на
32-битной архитектуре

# --- Проверка условия: i != n+1 ---
movl    %eax, %ecx              # Копируем i_low в %ecx
movl    %edx, %ebx              # Копируем i_high в %ebx
xorl    %esi, %ecx              # XOR с n_low (проверка равенства: если
равно, результат 0)
xorl    %edi, %ebx              # XOR с n_high
orl %ebx, %ecx                  # Объединяем результаты: если оба 0, то i ==
n+1
je  L1                          # Jump if Equal: если i == n+1, выходим из
цикла

# ТЕЛО ЦИКЛА (общая часть для чётного и нечётного i)
L5:
# --- Конвертация 64-битного i в float через SSE ---
movd    %eax, %xmm0             # i_low → младшие 32 бита %xmm0
                                # movd = Move Doubleword (32 бита)
movd    %edx, %xmm1             # i_high → младшие 32 бита %xmm1
movl    %eax, %ecx              # Копируем i_low для проверки чётности
xorl    %ebx, %ebx              # Обнуляем %ebx
punpckldq   %xmm1, %xmm0        # Упаковываем две 32-битные части в один
64-битный блок
                                # punpckldq = Packed Unpack Low Doubleword
                                # Теперь %xmm0 содержит полное 64-битное
значение i
andl    $1, %ecx                # i_low & 1 (проверка чётности: младший
бит)
```

```
movl    %ebx, 4(%esp)               # Сохраняем 0 в стек (для проверки 64-
битного результата)
movq    %xmm0, 8(%esp)              # Сохраняем 64-битное i в стек (для
загрузки в x87)
                                    # movq = Move Quadword (64 бита)
fildq   8(%esp)                     # Загружаем 64-битное целое из памяти в
x87 как float
                                    # fildq = Float Load Integer Quadword
                                    # Автоматически конвертирует int64 → 80-
бит float
movl    %ecx, (%esp)                # Сохраняем результат проверки чётности в
стек
movl    (%esp), %ecx                # Загружаем обратно (странная оптимизация
компилятора)
orl 4(%esp), %ecx                 # Проверяем: чётное или нечётное (объединяем
32-битные части)

# --- Вычисление 2*i + 1 ---
fld %st(0)                        # Дублируем st(0): теперь st(0)=i, st(1)=i
faddp   %st, %st(1)                 # st(1) = st(1) + st(0) = 2*i, pop
fld1                                # Загружаем константу 1.0 в st(0)
fadd    %st, %st(1)                 # st(1) = st(1) + st(0) = 2*i + 1

# --- Вычисление 1.0 / (2*i + 1) ---
fdivp   %st, %st(1)                 # st(1) = st(1) / st(0) = 1.0 / (2*i+1),
pop

# --- Умножение на 4.0 ---
flds    LC2                         # Загружаем константу 4.0 (float, 32 бита)
                                    # flds = Float Load Single
fmulp   %st, %st(1)                 # st(1) = st(1) * st(0) = 4.0 / (2*i+1),
pop
                                    # fmulp = Float Multiply and Pop

# --- Ветвление по чётности ---
je  L10                           # Jump if Equal: если чётное (результат
проверки = 0),
                                    # переход к ветке сложения (L10)

# ВЕТКА НЕЧЁТНОГО i (вычитание): pi -= 4 * (1.0 / (2.0*i + 1.0))
fsubrp  %st, %st(1)                 # st(1) = st(1) - st(0) = pi - вычисленное
                                    # fsubrp = Float Subtract Reverse and Pop
                                    # Результат в st(0) = новое значение pi
jmp L4                            # Переход к шагу цикла (i++)

# ОБРАБОТКА ОШИБКИ: n < 0 или выход из цикла
```

```
.p2align 4,,10
.p2align 3
L6:
fldz                                # Загружаем 0.0 в st(0) (возвращаем 0 при
ошибке)

L1:
# --- ЭПИЛОГ ФУНКЦИИ ---
leal    -12(%ebp), %esp             # Восстанавливаем указатель стека
popl    %ebx                        # Восстанавливаем callee-saved регистры
.cfi_restore 3
popl    %esi
.cfi_restore 6
popl    %edi
.cfi_restore 7
popl    %ebp
.cfi_restore 5
.cfi_def_cfa 4, 4
ret                                 # Возврат (результат в st(0))
.cfi_endproc
LFE4663:

# ФУНКЦИЯ main с инлайнгом sumc (32-бит, максимальная оптимизация)
.section    .text.startup, "x"
.p2align 4
.globl  _main
.def    _main;  .scl    2;  .type   32; .endef
_main:
LFB4664:
.cfi_startproc
# --- ПРОЛОГ ФУНКЦИИ ---
leal    4(%esp), %ecx               # Сохраняем старый %esp для отладки
.cfi_def_cfa 1, 0
andl    $-8, %esp                   # Выравнивание стека по 8 байт
pushl   -4(%ecx)
pushl   %ebp
movl    %esp, %ebp                  # Устанавливаем базовый указатель
.cfi_escape 0x10,0x5,0x2,0x75,0
pushl   %edi                        # Сохраняем callee-saved регистры
pushl   %esi
pushl   %ebx
pushl   %ecx
.cfi_escape 0xf,0x3,0x75,0x70,0x6
.cfi_escape 0x10,0x7,0x2,0x75,0x7c
.cfi_escape 0x10,0x6,0x2,0x75,0x78
.cfi_escape 0x10,0x3,0x2,0x75,0x74
subl    $36, %esp                   # Выделяем 36 байт локальной памяти
```

```
call    ___main                     # Инициализация runtime

# --- std::cin >> n; ---
leal    -24(%ebp), %eax             # Вычисляем адрес переменной n (64 бита)
movl    $__ZSt3cin, %ecx            # Загружаем адрес std::cin
movl    %eax, (%esp)                # Помещаем адрес n в стек (аргумент для
operator>>)
call    _ZNSi10_M_extractIxEERSiRT  # Вызов operator>>(long long&)
                                    # Результат: n сохранено по адресу -
24(%ebp)

# --- Загрузка n и проверка знака ---
movl    -20(%ebp), %edi             # n_high → %edi
movl    -24(%ebp), %esi             # n_low → %esi
subl    $4, %esp                    # Корректировка стека (stdcall)
testl   %edi, %edi                  # Проверяем знак n
js  L16                           # Если n < 0, переход к обработке ошибки

# --- Инициализация цикла (инлайнг sumc в main) ---
addl    $1, %esi                    # n_low++ (оптимизация условия)
fldz                                # pi = 0 в st(0)
adcl    $0, %edi                    # n_high += carry
xorl    %eax, %eax                  # i_low = 0
xorl    %edx, %edx                  # i_high = 0
jmp L15                           # Переход к телу цикла

# Выравнивание
.p2align 4,,10
.p2align 3

# --- Ветка ЧЁТНОГО (сложение) ---
L19:
faddp   %st, %st(1)                 # pi += вычисленное

# --- ШАГ ЦИКЛА ---
L14:
addl    $1, %eax                    # i_low++
adcl    $0, %edx                    # i_high += carry

# --- Проверка условия ---
movl    %eax, %ecx
movl    %edx, %ebx
xorl    %esi, %ecx                  # XOR с n_low
xorl    %edi, %ebx                  # XOR с n_high
orl %ebx, %ecx                    # Объединяем
je  L12                           # Выход если i == n+1
```

```
# --- ТЕЛО ЦИКЛА ---
L15:
movd    %eax, %xmm0                 # i_low → %xmm0
movd    %edx, %xmm1                 # i_high → %xmm1
movl    %eax, %ecx                  # Копируем для проверки чётности
xorl    %ebx, %ebx
punpckldq   %xmm1, %xmm0            # Упаковываем в 64-битный блок
andl    $1, %ecx                    # Проверка чётности
movl    %ebx, -36(%ebp)             # Сохраняем 0
movq    %xmm0, -32(%ebp)            # Сохраняем 64-битное i в стек
fildq   -32(%ebp)                   # Загружаем i в x87 как float
movl    %ecx, -40(%ebp)             # Сохраняем результат проверки
movl    -40(%ebp), %ecx             # Загружаем обратно
orl -36(%ebp), %ecx                # Проверяем чётность
fld %st(0)                         # Дублируем i
faddp   %st, %st(1)                 # 2*i
fld1                                # Загружаем 1.0
fadd    %st, %st(1)                 # 2*i + 1
fdivp   %st, %st(1)                 # 1.0 / (2*i+1)
flds    LC2                         # Загружаем 4.0
fmulp   %st, %st(1)                 # 4.0 * (...)
je  L19                            # Если чётное, переход к сложению
fsubrp  %st, %st(1)                 # Иначе вычитание (pi -= ...)
jmp L14                            # Переход к шагу цикла

# --- Обработка n < 0 ---
L16:
fldz                                # pi = 0

# --- Вывод результата ---
L12:
movl    __ZSt4cout, %eax            # Загружаем адрес std::cout
movl    $__ZSt4cout, %ecx           # Сохраняем адрес
movl    -12(%eax), %eax             # Доступ к внутреннему буферу cout
movl    $15, __ZSt4cout+4(%eax)     # setprecision(15) — инлайнинг!
                                    # Прямая запись значения 15 в поле объекта
cout
fstpt   (%esp)                      # Сохраняем pi в стек (аргумент для
operator<<)
call    _ZNSo9_M_insertIeEERSoT     # Вызов operator<<(long double) для вывода
pi

xorl    %eax, %eax                  # return 0
subl    $12, %esp                   # Корректировка стека
leal    -16(%ebp), %esp             # Восстанавливаем %esp
popl    %ecx                        # Восстанавливаем регистры
```

```
.cfi_restore 1
.cfi_def_cfa 1, 0
popl    %ebx
.cfi_restore 3
popl    %esi
.cfi_restore 6
popl    %edi
.cfi_restore 7
popl    %ebp
.cfi_restore 5
leal    -4(%ecx), %esp              # Восстанавливаем оригинальный %esp
.cfi_def_cfa 4, 4
ret                                 # Возврат
.cfi_endproc
LFE4664:

# --- Константа 4.0 (float, 32 бита) ---
.section .rdata, "dr"
.align 4
LC2:
.long   1082130432                  # IEEE 754: 0x40800000 = 4.0
```

x86-64 -m32 -O0

```
.file   "lab3.cpp"
.text

.file   "lab3.cpp"
.text
.file   "lab3.cpp"
.text
.file   "lab3.cpp"
.text

.file   "lab3.cpp"
.text

.section    .text$_ZSt12setprecisioni,"x"
.linkonce discard
.globl  __ZSt12setprecisioni
.def    __ZSt12setprecisioni;   .scl    2;  .type   32; .endef
__ZSt12setprecisioni:
```

```
LFB3476:
.cfi_startproc
pushl   %ebp                        # Пролог: сохраняем %ebp
.cfi_def_cfa_offset 8
.cfi_offset 5, -8
movl    %esp, %ebp                  # Устанавливаем базовый указатель
.cfi_def_cfa_register 5
movl    8(%ebp), %eax               # Загружаем аргумент (точность) из стека
                                    # 8(%ebp) = первый аргумент функции
(cdecl)
popl    %ebp                        # Эпилог: восстанавливаем %ebp
.cfi_restore 5
.cfi_def_cfa 4, 4
ret                                 # Возврат (результат в %eax)
.cfi_endproc
LFE3476:

.text

# ФУНКЦИЯ sumc
.globl  __Z4sumcx
.def    __Z4sumcx;  .scl    2;  .type   32; .endef
__Z4sumcx:
LFB4582:
.cfi_startproc
# --- ПРОЛОГ ФУНКЦИИ ---
pushl   %ebp                        # Сохраняем старый базовый указатель
.cfi_def_cfa_offset 8
.cfi_offset 5, -8
movl    %esp, %ebp                  # Устанавливаем новый базовый указатель
.cfi_def_cfa_register 5
andl    $-8, %esp                   # Выравнивание стека по 8 байт
subl    $32, %esp                   # Выделяем 32 байта локальной памяти

# --- Загрузка аргумента n (64-битное число в двух 32-битных частях) ---
movl    8(%ebp), %eax               # Младшие 32 бита n → %eax
movl    12(%ebp), %edx              # Старшие 32 бита n → %edx
movl    %eax, (%esp)                # Сохраняем n_low в стек (по адресу %esp)
movl    %edx, 4(%esp)               # Сохраняем n_high в стек (по адресу
%esp+4)
                                    # В 32-бит архитектуре long long
передаётся через стек

# --- Инициализация: long double pi = 0; ---
fldz                                # Загружаем 0.0 в st(0) (стек FPU)
fstpt   20(%esp)                    # Сохраняем pi = 0 по адресу 20(%esp)
```

```
                                    # fstpt = Float Store Ten-byte and Pop
(80-бит long double)

# --- Инициализация цикла: for (long long i = 0; ...) ---
movl    $0, 8(%esp)                 # i_low = 0 (по адресу %esp+8)
movl    $0, 12(%esp)                # i_high = 0 (по адресу %esp+12)
                                    # Переменная i хранится в стеке (без
оптимизаций)
jmp L4                             # Переход к проверке условия цикла

# ТЕЛО ЦИКЛА
L7:
# --- Проверка чётности: if (i % 2 == 0) ---
movl    8(%esp), %eax               # Загружаем i_low из стека
movl    12(%esp), %edx              # Загружаем i_high из стека
andl    $1, %eax                    # i_low & 1 (вычисляем i % 2)
movl    $0, %edx                    # Обнуляем старшую часть (для 64-битной
проверки)
movl    %eax, %ecx                  # Копируем результат
orl %edx, %ecx                     # Объединяем: проверяем, равен ли результат
0
jne L5                             # Jump if Not Equal: если нечётное, переход
к else (L5)

# ВЕТКА IF (i чётное): pi += 4 * (1.0 / (2.0*i + 1.0))
fildq   8(%esp)                     # Загружаем 64-битное целое i из стека в
x87
                                    # fildq = Float Load Integer Quadword
                                    # Автоматически конвертирует int64 → 80-
бит float
fld %st(0)                         # Дублируем st(0): теперь st(0)=i, st(1)=i
faddp   %st, %st(1)                 # st(1) = st(1) + st(0) = 2*i, pop
fld1                                # Загружаем константу 1.0 в st(0)
faddp   %st, %st(1)                 # st(1) = st(1) + st(0) = 2*i + 1, pop
fld1                                # Загружаем 1.0 в st(0)
fdivp   %st, %st(1)                 # st(1) = st(1) / st(0) = 1.0 / (2*i+1),
pop
fldt    LC2                         # Загружаем константу 4.0 (80-бит long
double)
                                    # fldt = Float Load Ten-byte
fmulp   %st, %st(1)                 # st(1) = st(1) * st(0) = 4.0 / (2*i+1),
pop
                                    # fmulp = Float Multiply and Pop

# --- Сложение с pi: pi += вычисленное значение ---
fldt    20(%esp)                    # Загружаем текущее значение pi из стека
```

```
faddp   %st, %st(1)                 # st(1) = st(1) + st(0) = pi +
вычисленное, pop
fstpt   20(%esp)                    # Сохраняем новое значение pi обратно в
стек
                                    # fstpt = Float Store Ten-byte and Pop
jmp L6                             # Переход к шагу цикла (пропускаем ветку
else)

# ВЕТКА ELSE (i нечётное): pi -= 4 * (1.0 / (2.0*i + 1.0))
L5:
# --- Вычисление 4.0 / (2.0*i + 1.0) ---
fildq   8(%esp)                     # Загружаем i
fld %st(0)                         # Дублируем
faddp   %st, %st(1)                 # 2*i
fld1                                # 1.0
faddp   %st, %st(1)                 # 2*i + 1
fld1                                # 1.0
fdivp   %st, %st(1)                 # 1.0 / (2*i+1)
fldt    LC2                         # 4.0
fmulp   %st, %st(1)                 # 4.0 / (2*i+1)

# --- Вычитание из pi: pi -= вычисленное значение ---
fldt    20(%esp)                    # Загружаем текущее pi
fsubp   %st, %st(1)                 # st(1) = st(1) - st(0) = pi -
вычисленное, pop
                                    # fsubp = Float Subtract and Pop
fstpt   20(%esp)                    # Сохраняем обновлённое pi

# ШАГ ЦИКЛА: i++ (64-битный инкремент через пару ячеек стека)
L6:
addl    $1, 8(%esp)                 # i_low++ (инкремент младшей части в
стеке)
adcl    $0, 12(%esp)                # i_high += carry (инкремент старшей части
с переносом)
                                    # adcl = Add with Carry Long

# ПРОВЕРКА УСЛОВИЯ ЦИКЛА: i <= n
L4:
movl    8(%esp), %eax               # Загружаем i_low
movl    12(%esp), %edx              # Загружаем i_high
movl    4(%esp), %ecx               # Загружаем n_high (из стека, адрес
%esp+4)
cmpl    %eax, (%esp)                # Сравниваем n_low и i_low
                                    # cmpl = Compare Long (вычисляет n_low -
i_low)
sbbl    %edx, %ecx                  # Вычитаем i_high из n_high с учётом займа
                                    # sbbl = Subtract with Borrow Long
```

```
                                    # Это эмуляция 64-битного сравнения на 32-
битной архитектуре
jge L7                              # Jump if Greater or Equal: если n >= i,
продолжаем цикл

# ВОЗВРАТ РЕЗУЛЬТАТА: return pi;
fldt    20(%esp)                    # Загружаем финальное значение pi в st(0)
                                    # В x87 возврат из функции ВСЕГДА
происходит через st(0)
leave                               # Эпилог: эквивалент movl %ebp, %esp; popl
%ebp
.cfi_restore 5
.cfi_def_cfa 4, 4
ret                                 # Возврат (результат уже в st(0))
.cfi_endproc
LFE4582:

# ФУНКЦИЯ main
.globl  _main
.def    _main;  .scl    2;  .type   32; .endef
_main:
LFB4583:
.cfi_startproc
# --- ПРОЛОГ ФУНКЦИИ ---
pushl   %ebp                        # Сохраняем %ebp
.cfi_def_cfa_offset 8
.cfi_offset 5, -8
movl    %esp, %ebp                  # Устанавливаем базовый указатель
.cfi_def_cfa_register 5
subl    $32, %esp                   # Выделяем 32 байта локальной памяти
call    ___main                     # Инициализация runtime среды MinGW

# --- std::cin >> n; ---
leal    -20(%ebp), %eax             # Вычисляем адрес переменной n (64 бита)
                                    # leal = Load Effective Address Long
movl    %eax, (%esp)                # Помещаем адрес n в стек (аргумент для
operator>>)
movl    $__ZSt3cin, %ecx            # Загружаем адрес std::cin в %ecx
                                    # Нестандартное расширение GCC: передача
this через %ecx
call    __ZNSirsERx                 # Вызов operator>>(long long&)
                                    # Результат: n сохранено по адресу -
20(%ebp)
subl    $4, %esp                    # Корректировка стека (stdcall очистка)

# --- Вызов функции sumc(n) ---
movl    -20(%ebp), %eax             # Загружаем n_low
```

```
movl    -16(%ebp), %edx             # Загружаем n_high
movl    %eax, (%esp)                # Помещаем n_low в стек (аргумент 1)
movl    %edx, 4(%esp)               # Помещаем n_high в стек (аргумент 2)
call    __Z4sumcx                   # Вызов функции sumc(n)
                                    # Результат возвращается в st(0)
fstpt   -12(%ebp)                   # Сохраняем результат (pi) в локальную
переменную

# --- std::cout << std::setprecision(15) << pi; ---
movl    $15, (%esp)                 # Помещаем аргумент 15 в стек
call    __ZSt12setprecisioni        # Вызов функции setprecision(15)
                                    # Результат (манипулятор) в %eax
movl    %eax, 4(%esp)               # Помещаем манипулятор в стек (аргумент 2)
movl    $__ZSt4cout, (%esp)         # Помещаем адрес std::cout в стек
(аргумент 1)
call
__ZStlsIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_St13_Setprecision
                                    # Вызов operator<<(манипулятор) для
установки точности

# --- Вывод значения pi ---
fldt    -12(%ebp)                   # Загружаем pi (long double) из памяти в
st(0)
fstpt   (%esp)                      # Сохраняем pi в стек (аргумент для
operator<<)
movl    %eax, %ecx                  # Сохраняем поток cout в %ecx
call    __ZNSolsEe                  # Вызов operator<<(long double) для вывода
pi
subl    $12, %esp                   # Корректировка стека

# --- Возврат из main ---
movl    $0, %eax                    # return 0 (код успешного завершения)
leave                               # Эпилог: восстанавливаем %esp и %ebp
.cfi_restore 5
.cfi_def_cfa 4, 4
ret                                 # Возврат
.cfi_endproc
LFE4583:

# СЕКЦИЯ ДАННЫХ: константа 4.0 (80-бит long double)
.section .rdata, "dr"
.align 16
LC2:
.long   0                           # Мантисса (младшие 32 бита)
.long   -2147483648                 # Мантисса (старшие 32 бита): 0x80000000
.long   16385                       # Экспонента и знак: 0x4001 → 2^2 * 1.0 =
4.0
```

```
                                # Формат 80-бит long double: 1 бит знака,
15 бит экспоненты, 64 бита мантиссы
```

5. Заключение:

В процессе выполнения лабораторной работы я изучил архитектурные особенности процессоров x86 и x86-64, а также получил практическое представление о том, как происходит трансляция программ с языка C++ в машинные инструкции ассемблера. В ходе анализа ассемблерного листинга программы из предыдущей лабораторной работы удалось проследить, каким образом компилятор распределяет переменные по ячейкам памяти и регистрам процессора при использовании различных уровней оптимизации кода. Применение оптимизации позволило существенно уменьшить число операций чтения и записи в память, что положительно сказывается на быстродействии программы. Итогом работы стало приобретение навыков чтения и анализа ассемблерного кода, углубление понимания принципов работы компилятора, а также формирование представления о том, как происходит выполнение программ на аппаратном уровне.

6. Команды для компиляции :

g++ -S -O0 lab3.cpp -o lab3_O0.s

g++ -S -O1 lab3.cpp -o lab3_O3.s

g++ -S -O2 lab3.cpp -o lab3_Og.s

g++ -S -m32 -O0 lab3.cpp -o a_m32_O0.s

g++ -S -m32 -O1 lab3.cpp -o lab3_m32_O3.s